

EXAMEN DE SYNTHÈSE

Nom étudiant : Chetari Aya

Cible DE0-Nano (Cyclone IV)	Outil Quartus II 13.0	Durée 2H 15h30-17h30	Total 42 points Note : _____ / 20
------------------------------------	------------------------------	--	---

Consignes générales

- Documentation autorisée.
- Module fourni : **Aucun**
- Répondez directement sur ce sujet dans les espaces prévus.
- Précision des terminologie utilisée (VHDL, SPI, codesign, ...).
- Le barème est indiqué entre crochets après chaque question.
- Les questions sont indépendantes. En cas de blocage, passez à la suivante.
- **Vos réponses doivent être courtes, concises et donc précises.**
- **Votre projet, vos descriptions doivent être bien indentés, structurés et commentés avec un juste nécessaire.**
- **Vos versions antérieures (Implémentations TP) peuvent être utilisées.**
- **Version propre des projets remis**
- Rendu sous forme d'un projet GITHUB incluant ce document contenant les réponses aux questions ; **dernière heure** de mise à jour **(20 Mai 2026 – 17h35).**
- **Pénalité(s) de remise en retard ou (et) de post-modification (toute post-modification de n'importe quel fichier annule sa prise en compte).**

Partie 1 — Architecture et Généralités sur les SoCs (6 points)

Q1. Définissez la différence fondamentale entre un 'Soft IP Core' et un 'Hard IP Core'. Citez un avantage et un inconvénient pour chaque approche. Etayer votre réponse avec un exemple concret dans chaque cas **[1.5 pts]**

Réponse :

- **Soft IP** : code HDL synthétisable, mappé sur les ressources programmables du FPGA à la compilation.
 - **Avantage** : flexible et portable (paramétrable, réutilisable d'un FPGA à l'autre).
 - **Inconvénient** : moins rapide et plus gourmand en ressources logiques.
- **Hard IP** : bloc gravé en silicium dans le FPGA, figé dès la fabrication.

Avantage : très haute performance, faible consommation, n'utilise pas la logique programmable.

Inconvénient : figé, non modifiable, dépend du FPGA

.....

Q2. Quel est le rôle de la couche d'Abstraction Matérielle (HAL) dans l'écosystème du NIOS II ?
Comment facilite-t-elle la portabilité du code ? **[1.5 pts]**

Réponse :

La **HAL** est une couche logicielle générée par le BSP qui fournit une API standard (IORD, IOWR, printf...) pour accéder aux périphériques sans manipuler directement les adresses physiques.

Elle **facilite la portabilité** car le code utilise des noms symboliques (PWM_BASE) définis dans system.h : si l'adresse ou le matériel change dans Qsys, il suffit de régénérer le BSP sans modifier l'application

.....

.....

.....

Q3. Pourquoi l'utilisation de bus physiques partagés uniques (avec buffers 3-états) est-elle aujourd'hui abandonnée dans les SoC modernes contenant des dizaines d'IPs au profit de Bus multiplexés ou de NoC ? **[1.5 pts]**

Réponse :

Les bus 3-états partagés ont été abandonnés parce qu'ils sont lents (capacité parasite qui augmente avec chaque IP), non synthétisables en interne sur FPGA/ASIC modernes, dangereux (risque de contention destructive) et non parallélisables (une seule transaction à la fois).

Les bus multiplexés (Avalon, AXI) les remplacent avec des mux purement CMOS, sans contention et avec un timing propre. Les NoC vont plus loin en routant les transactions comme des paquets sur un réseau, permettant des dizaines de transactions simultanées indispensable pour les SoC modernes à 100+ IPs

.....

.....

.....

Q4. Expliquez le principe d'une transaction décomposée sur un système d'interconnexion. En quoi améliore-t-elle la bande passante par rapport à une transaction bloquante ? [1.5 pts]

Réponse :

Dans une transaction bloquante, le maître envoie une requête et occupe le bus jusqu'à recevoir la réponse pendant ce temps personne d'autre ne peut l'utiliser.

Dans une transaction décomposée, la requête et la réponse sont séparées : le maître envoie sa requête, libère le bus, et l'esclave renvoie la réponse plus tard via un canal séparé (identifiée par un tag).

Gain de bande passante

Pendant que l'esclave traite, le bus est libre pour d'autres transactions. On peut aussi avoir plusieurs requêtes en vol en même temps, ce qui sature mieux le bus au lieu d'attendre.

Partie 2 — Analyse de Performance d'une Architecture de Von Neumann (5 points)

Q5. Soit un processeur Von Neumann 16 bits muni d'un bus d'adresses de 24 bits et cadencé à 2,4 GHz. Précisez les registres à mettre en oeuvre pour accéder à cet espace mémoire en déterminant sa capacité directement adressable (en octets, Mo et Go) ainsi que la durée exacte d'un cycle d'horloge (en ns). [1.5 pts]

Capacité adressable

$2^{24} = 16\,777\,216$ adresses, chacune sur 16 bits (2 octets) :

$$2^{24} \times 2 = 2^{25} = 33\,554\,432 \text{ octets} = 32 \text{ Mo} \approx 0,031 \text{ Go}$$

Registres

- **MAR** : 24 bits (adresse)
- **MDR** : 16 bits (donnée)
- **PC** : 24 bits
- **IR** : 16 bits

Durée d'un cycle

$$T = \frac{1}{2,4 \times 10^9} \approx 0,417 \text{ ns}$$

Q6. On traite un volume de données de 128 MiB organisé en blocs de 8 octets. La lecture d'un bloc nécessite 7 cycles et son traitement 9 cycles. Calculez le nombre de blocs et le temps total requis en millisecondes (ms) pour traiter toutes ces données. [1.5 pts]

Réponse :

Nombre de blocs

$$128 \text{ MiB} = 128 \times 2^{20} = 134\,217\,728 \text{ octets}$$

$$\text{Nb blocs} = \frac{134\,217\,728}{8} = 16\,777\,216 \text{ blocs} = 2^{24}$$

Cycles totaux

Par bloc : $7 + 9 = 16$ cycles.

$$\text{Total} = 16\,777\,216 \times 16 = 268\,435\,456 \text{ cycles} = 2^{28}$$

Temps total

Avec $T \approx 0,4167\text{ns/cycle}$:

$$268\,435\,456 \times 0,4167 \times 10^{-9} \approx 0,1118 \text{ s} \approx \mathbf{111,8 \text{ ms}}$$

Q7. Un second processeur cadencé à 1,3 GHz lit un bloc de 4 octets en 2 cycles et le traite en 5 cycles. Calculez le débit de traitement effectif (en Mo/s) des deux processeurs. Lequel est le plus rapide ? Justifiez par le calcul. **[2 pts]**

Réponse :

Processeur 1 (2,4 GHz, blocs de 8 octets, 7+9 = 16 cycles)

$$\text{Débit}_1 = \frac{8 \text{ octets}}{16 \text{ cycles}} \times 2,4 \times 10^9 \text{ cycles/s} = 1,2 \times 10^9 \text{ o/s} = \mathbf{1200 \text{ Mo/s}}$$

Processeur 2 (1,3 GHz, blocs de 4 octets, 2+5 = 7 cycles)

$$\text{Débit}_2 = \frac{4 \text{ octets}}{7 \text{ cycles}} \times 1,3 \times 10^9 \approx 7,43 \times 10^8 \text{ o/s} \approx \mathbf{743 \text{ Mo/s}}$$

Le processeur 1 est plus rapide que le processeur 2

Partie 3 — Programmation Bas Niveau et Intégration sous Qsys (6,5 points)

Q8. Analysez le code d'ouverture de flux HAL suivant :

```
fp = fopen("/dev/uart1", "w");  
fprintf(fp, "hello");
```

Expliquez comment la couche HAL parvient à lier une fonction d'I/O standard du C à un périphérique physique mappé en mémoire. **[1.5 pts]**

Réponse :

fopen("/dev/uart1", "w") : la HAL maintient une table des périphériques (alt_dev_list) qui associe chaque nom /dev/xxx à un driver (pointeurs de fonctions read/write) et à son adresse de base définie dans system.h. fopen retourne un FILE* qui encapsule ces infos.

fprintf(fp, "hello") : fprintf formate la chaîne puis appelle write(), que la HAL redirige vers le driver UART via le pointeur de fonction. Le driver finit par exécuter IOWR_ALTERA_AVALON_UART_TXDATA(UART1_BASE, c) pour chaque caractère.

Q9. Dans le cadre de l'utilisation d'une Custom Instruction combinatoire, décrivez comment ce bloc logique personnalisé est physiquement raccordé au processeur. Quel est son temps d'exécution en cycles d'horloge ? [2 pts]

Réponse :

Une Custom Instruction combinatoire est un bloc logique inséré directement dans l'ALU du Nios II. Elle est raccordée via une interface dédiée comportant :

- dataa, datab (32 bits chacun) : les deux opérandes, venant du banc de registres,
- result (32 bits) : la sortie, recâblée vers le banc de registres,
- n : un index qui sélectionne quelle custom instruction est utilisée .

Temps d'exécution

1 seul cycle d'horloge comme une instruction ALU native

Q9.Bis. Est-ce qu'il est envisageable via cette instruction personnalisée de piloter l'acquisition des données capteurs du CUTE CAR. Justifiez votre réponse ? [2 pts]

.....
.....
.....
.....

Q10. On souhaite maintenant réaliser cette même fonction d'accélération sous la forme d'un composant matériel personnalisé (Custom Component) autonome câblé sur le bus Avalon. Énumérez et expliquez le rôle des signaux fondamentaux de l'interface Avalon Memory-Mapped (Avalon-MM) nécessaires pour permettre au NIOS II de lire et d'écrire des données dans les registres internes de ce nouveau périphérique. En donner la justification. [3 pts]

Réponse :

Signaux interface avalon MM

Clk : horloge commune Nios II vers périphérique.

reset_n : initialise les registres internes.

address : sélectionne le registre à accéder.

write : commande d'écriture du Nios vers le périphérique.

writedata : donnée à écrire (32 bits).

read : commande de lecture du périphérique par le Nios

readdata : donnée lue depuis le périphérique (32 bits).

Justification

Ces 7 signaux permettent au Nios II d'écrire dans les registres internes (via address + write + writedata) et de lire les résultats (via address + read + readdata). C'est le minimum qu'il faut avoir pour que le processeur accède au composant personnalisé comme à une mémoire adressée.

On peut aussi avoir aussi :

chipselect : active le périphérique quand le Nios accède à sa plage d'adresses (valide les opérations read/write).

byteenable (4 bits) : indique quels octets du mot 32 bits sont concernés (permet des accès partiels : 8, 16 ou 32 bits).

Adresse : si on utilise plusieurs registres pour pouvoir sectionner le registre pour écrire /lire

Partie 4 — Synthèse globale & Arbitrage Co-design (2,5 points)

Q11. Arbitrage Technologique : Comparez de manière critique l'implémentation d'une fonction d'accélération algorithmique sous forme de 'Custom Instruction' (ALU) par rapport à une implémentation sous forme de 'Custom Component Qsys' (Périphérique Avalon-MM). Développez votre argumentation selon les trois critères suivants :

- a) Latence d'accès et vitesse d'exécution pure de la fonction.
- b) Impact sur l'encombrement et le trafic du réseau d'interconnexion (système de bus).
- c) Capacité d'évolution. **[2.5 pts]**

Réponse :

La Custom Instruction offre une latence minimale d'1 cycle , idéale pour les petites fonctions critiques , mais elle est figée dans le cœur Nios II et difficile à modifier ou porter. Le Custom Component Avalon-MM introduit une latence de 3-4 cycles et consomme du trafic bus , mais il bénéficie d'une logique dédiée qui ne pollue pas le processeur, d'une grande flexibilité via les registres de configuration, et d'une réutilisabilité auprès de tout maître Avalon.

Partie 5 — Implémentation Custom Component Avalon-MM (15 points)

On souhaite concevoir un composant Qsys personnalisé effectuant une inversion des octets d'un mot de 32 bits. Le composant est connecté via une interface Avalon Memory-Mapped (Slave). Le mot 32 bits est structuré ainsi : [Octet3 | Octet2 | Octet1 | Octet0]. Le résultat attendu au choix est : [Octet0 | Octet1 | Octet2 | Octet3] ou [Octet1 | Octet0 | Octet3 | Octet2]

Q12. Proposez l'architecture VHDL du module de calcul combinatoire. **[2 pts]**

Réponse :

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity byte_inverter is
  port(
    input : in std_logic_vector(31 downto 0);
    output : out std_logic_vector(31 downto 0);
    mode : in std_logic -- 0 = inversion complète, 1 = inversion par paires
  );
end entity;

architecture combinatoire of byte_inverter is
begin

  output <=
    input(7 downto 0) & input(15 downto 8) & input(23 downto 16) & input(31 downto 24)
    when mode = '0' else
    input(15 downto 8) & input(7 downto 0) & input(31 downto 24) & input(23 downto 16);

end combinatoire;
```

Q13. Listez les interfaces avalon et les signaux de l'interface nécessaires pour que le processeur NIOS II puisse écrire un mot dans le registre du composant et lire le résultat '. [3 pts]

Réponse :

clk : horloge de synchronisation
reset_n : reset asynchrone actif bas
write : indique une écriture du Nios II
read : indique une lecture du Nios II
address : sélectionne le registre (1 bit suffit pour 2 registres : entrée et résultat)
writedata (32 bits) : mot à inverser, envoyé par le Nios II
readdata (32 bits) : mot inversé, renvoyé au Nios II

Q13.Bis Identifiez le ou les type(s) d'interface sur les chronogrammes suivants. Expliquez les séquences d'échanges mises en oeuvre [3 pts]

Type d'interface

Avalon Memory-Mapped (Avalon-MM) — interface maître/esclave synchrone pour accès à la mémoire ou aux périphériques.

Chronogramme 1 (vue maître)

Séquence :

- t_0-t_1 : maître pose address et writedata, asserte write → écriture dans l'esclave.
- t_1-t_2 : maître libère write, pose une nouvelle address.
- t_2-t_3 : maître asserte read → démarre une lecture.
- t_3-t_4 : esclave pose readdata sur le bus, maître reçoit la donnée.

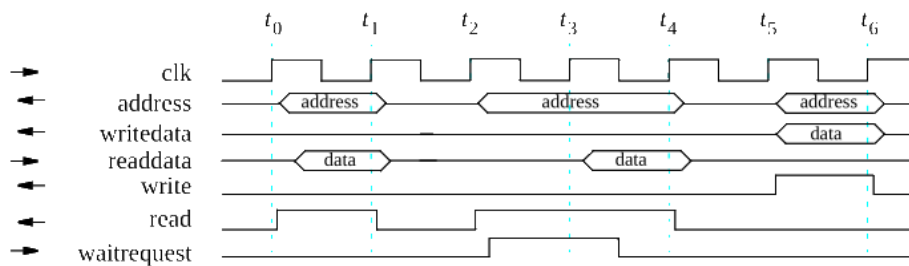
- t_4-t_5 : maître lance une nouvelle écriture.

Le waitrequest du maître reste bas = pas d'attente, transactions en 1 cycle.

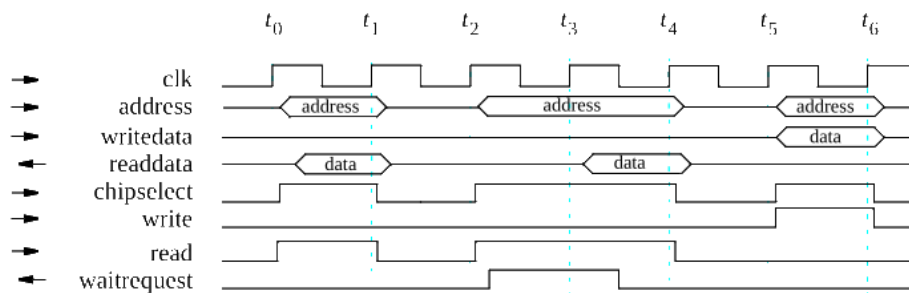
Chronogramme 2 (vue esclave)

Mêmes transactions vues du côté esclave :

- t_0-t_1 : chipselect = 1, write = 1 → esclave reçoit et mémorise writedata.
- t_1-t_2 : write redescend, chipselect remonte brièvement.
- t_2-t_3 : chipselect = 1, read = 1 → esclave prépare readdata.
- t_3-t_4 : readdata valide, maître l'échantillonne.
- t_4-t_6 : cycles d'attente (waitrequest = 1) pendant que l'esclave traite.



Timing diagram for read/write transactions from the master's point of view.



Timing diagram for read/write transactions from the slave's point of view.

Q14. Implémenter cet IP core et le valider sous une approche HAL dans un projet Quartus. Copie d'écran des résultats obtenus [10 pts]

Réponse :

.....

.....

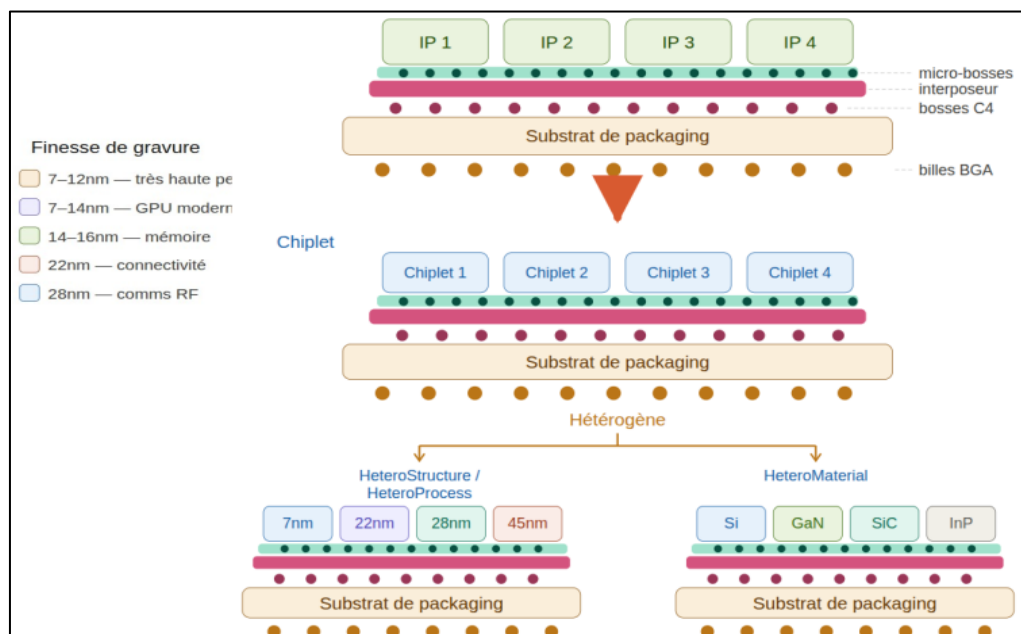
.....

.....

.....

Partie 6 — Evolution des SoCs (5 points)

Q15. Définissez le concept de Chiplet et expliquez en deux arguments pourquoi l'industrie des semi-conducteurs a progressivement abandonné l'intégration SoC monolithique pour cette approche. Illustrez votre réponse en vous appuyant sur l'illustration suivante [5 pts]:



Définition

Un Chiplet est une petite puce spécialisée qui communique avec d'autres via un substrat d'interconnexion commun. Exemple : CPU sur un chiplet, GPU sur un autre, mémoire sur un troisième — tous reliés et formant un système complet.

Pourquoi abandonner le SoC monolithique ?

Argument 1 : Coût de fabrication

- Un gros die à 5 nm coûte très cher et a peu de rendement (un défaut = tout le die perdu).
- Avec chiplets : on grave le CPU à 5 nm (nécessaire) et l'I/O à 28 nm (ancien processus, pas cher). Rendement meilleur, coût total réduit.

Argument 2 : Réutilisabilité et flexibilité

- Un chiplet CPU conçu une fois peut équiper plusieurs produits/générationes différentes.
- Les briques matérielles deviennent modulables et interchangeable, pas besoin de tout redévelopper à chaque fois.

Tableau Récapitulatif du Barème

Partie	Thématique de l'Évaluation	Concepts Clés Validés	Points
Partie 1	Généralités & Architectures SoC	IP Cores, Couche d'abstraction HAL, Topologies de Bus vs NoC, Split transactions	6.0 pts
Partie 2	Performances Architecturales	Espace d'adressage, Fréquence, Cycles, Calculs de temps et Débits d'octets	5.0 pts
Partie 3	Intégration Matérielle & Qsys	Flux HAL/UART, Extensions d'ALU, Signaux de l'interface Avalon Memory-Mapped (Avalon-MM)	8.5 pts

Partie 4	Synthèse & Arbitrage Co-design	Analyse critique et critères de choix : Custom Instruction vs Composant Qsys Avalon	2.5 pts
Partie 5	Custom Component Avalon-MM (Swap Octets)	Architecture VHDL swap_octets, Signaux interface Avalon-MM Slave	18.0 pts
Partie 6	Evolution des SoCs	Soc Hétérogènes - Chiplet	5.0 pts
TOTAL	Examen de Synthèse Global	Validation des compétences Soc, d'architecture, co-design, programmation haut et bas niveau	45 pts